

به نام ایزد یکتا

برنامه سازی پیشرفته - 4041

استاد درس : آقای دکتر علی جاویدانی

اعضای تیم دستیاران دانشجویی : مهدیس یعقوب انصاری - کیانا جابری - مبینا صفری متین امیرپناه فر -
شهاب ثمری شمس - محمد متین سلیمانی - پارسا دولت آبادی - نیما مخملی



سلام بچه ها ! 🙌

به مینی پروژه ی سوم خیلی خوش اومدین ... 😊

توی این ایستگاه قراره وارد دنیایی بشیم که هر کلاس یک کاراکتره، هر متد یک تصمیم، و طراحی درست، تفاوت بین یک بازی خوب و یک کد شلوغ رو رقم می‌زنه.

این تمرین شامل سه بخش مستقل است.

هدف از این تمرین، سنجش توانایی شما در تحلیل مسئله، طراحی معماری شیء‌گرا و استفاده صحیح از مفاهیم پیشرفته زبان C++ می‌باشد.

در هر بخش، تمرکز اصلی بر چیهستی مسئله و الزامات طراحی است و نه تحمیل یک شیوه خاص برای پیاده‌سازی. اما قالب اصلی خواسته شده باید دقیق و مطابق سند پیاده سازی شود. انتخاب ساختار کلاس‌ها، نحوه تعامل اشیاء و جزئیات طراحی، بخشی از فرایند ارزیابی شما محسوب می‌شود.

سال‌ها دل طلب جام جم از ما می‌کرد ... و آنچه خود داشت ز بیگانه تمنّا می‌کرد

دیدمش خُرّم و خندان قدح باده به دست ... و اندر آن آینه صد گونه تماشا می‌کرد

گفتم این جام جهان‌بین به تو کی داد حکیم؟

گفت آن روز که این گنبد مینا می‌کرد

تمرین سوم - بخش اول | Advanced Time & Resource Management System

در بسیاری از سامانه‌های نرم‌افزاری، مدیریت بهینه منابع مشترک در بازه‌های زمانی مختلف اهمیت بالایی دارد؛ از فضاهای آموزشی و آزمایشگاه‌ها گرفته تا سرورها، پردازنده‌ها و منابع محاسباتی. طراحی چنین سامانه‌هایی نیازمند مدلی دقیق، منعطف است.

این سیستم باید بتواند منابعی مانند کلاس درس، آزمایشگاه، اتاق جلسه، یا حتی منابع محاسباتی را مدل‌سازی کرده و زمان‌های در دسترس یا رزرو شده آن‌ها را مدیریت کند.

تمرکز پروژه بر طراحی شیء‌گرا، روابط درست بین کلاس‌ها، و استفاده معنادار از قابلیت‌های **C++** است، نه پیاده‌سازی یک سناریوی خاص.

اهداف اصلی سیستم

سیستم شما باید قابلیت‌های زیر را داشته باشد:

1. مدل‌سازی منابع (Resource)

- هر منبع دارای شناسه، نام و نوع مشخص است
- هر منبع می‌تواند چندین بازه زمانی مرتبط داشته باشد

2. مدیریت بازه‌های زمانی (TimeInterval)

- تعریف بازه با زمان شروع و پایان
- بررسی همپوشانی دو بازه
- محاسبه طول بازه
- مقایسه بازه‌ها

3. تحلیل و ترکیب بازه‌های زمانی

- تشخیص تداخل بین دو بازه
- ترکیب بازه‌های متوالی یا همپوشان
- مقایسه بازه‌ها با عملگرها

4. گزارش‌دهی ساخت‌یافته

- نمایش اطلاعات منابع
- نمایش بازه‌های زمانی هر منبع به‌صورت مرتب و خوانا

کلاس‌های الزامی

کلاس Time

نماینده یک نقطه زمانی

- ویژگی‌ها:

- hour ○
- minute ○

- وظایف:

- مقایسه زمان‌ها
- محاسبه اختلاف زمانی
- عملگرهای مورد انتظار (overloading):
 - (>)
 - (==)
 - (-) (اختلاف زمانی به دقیقه)

کلاس TimeInterval

نماینده یک بازه زمانی

- ویژگی‌ها:

- Time start ○
- Time end ○

- متدهای الزامی:

- `bool overlaps(const TimeInterval&)`
- `int duration()`
- `TimeInterval merge(const TimeInterval&)`

- Operator Overloading الزامی (مغدادار):

- `Operator +` → ترکیب دو بازه قابل ادغام
- `Operator <` → مقایسه براساس زمان شروع
- `Operator ==` → برابر بودن دو بازه

کلاس Resource

نماینده یک منبع قابل مدیریت

- ویژگی‌ها:

- id
- name
- `vector<TimeInterval> intervals`

- متدهای الزامی:

- `addInterval(...)`
- `hasConflict(...)`
- `printSchedule()`

- نکات مهم طراحی:

- انتقال اشیاء به توابع با `reference`
- استفاده درست از `this`
- جلوگیری از تعارض بازه‌ها

کلاس ResourceManager

مدیر کل سامانه

- ویژگی‌ها:

- `vector<Resource> resources`
- `static int totalResources`

- وظایف:

- افزودن منبع جدید
- جستجوی منبع
- گزارش کلی سیستم

- استفاده الزامی از `static`:

- شمارنده کل منابع
- متدهای `utility` (در صورت نیاز)

نکات مهم

- این پروژه نباید hard-code شده باشد
- طراحی کلاس‌ها مهم‌تر از تعداد متدهاست
- Operator Overloading فقط در صورت توجیه منطقی
- خوانایی، انسجام، و مدل ذهنی سیستم اهمیت دارد

Criterion	Description	Points
System & OOP Design	Clear architecture, separation of responsibilities, scalability	20
Correctness & Logic	Resource and interval management works, conflict detection correct	20
Operator Overloading	Operators reflect problem logic, not just syntax	15
Function Overloading & Object Handling	Proper use of overloaded methods, pass by reference/const-ref	10
static & this	Meaningful use of static members, correct <code>this</code> usage / method chaining	10
Code Quality	Readability, formatting, naming, comments	10
Reporting & Output	Clear, structured schedule and resource reporting	15

تمرین سوم - بخش دوم | Smart Banking & Transaction Engine

سیستم های بانکی از هسته های ترین اجزای نرم افزارهای مالی محسوب می شوند و طراحی نادرست آنها می تواند منجر به خطاهای منطقی، امنیتی و عملیاتی شود. یک سیستم بانکی شیءگرا باید قادر باشد:

- حساب ها، تراکنش ها و مشتریان را به شکل ایمن مدیریت کند
- انواع مختلف تراکنش (واریز، برداشت، انتقال بین حساب ها) را پشتیبانی نماید
- تعامل بین اجزای سیستم از طریق اشیاء و عملگرهای پازنویسی شده انجام شود
- امکان گزارش گیری دقیق و ساخت یافته از وضعیت حساب ها و تراکنش ها فراهم شود

تمرکز پروژه بر استفاده پیشرفته از ++C، طراحی شیءگرا، و مدل سازی درست مفاهیم بانکی است، نه پیاده سازی یک بانک واقعی یا رابط کاربری کامل.

اهداف اصلی سیستم

سیستم شما باید قابلیت های زیر را داشته باشد:

مدیریت حساب ها (Account)

- هر حساب دارای شناسه، نام صاحب حساب، موجودی و نوع حساب است
- امکان واریز و برداشت امن موجودی
- بررسی محدودیت ها (مثلاً حداقل موجودی، محدودیت برداشت)

مدیریت تراکنش ها (Transaction)

- هر تراکنش دارای شناسه، نوع، مبلغ و زمان انجام است
- تراکنش ها باید قابلیت مقایسه، جمع و ترکیب منطقی داشته باشند
- استفاده از Operator Overloading برای تراکنش ها (مثلاً + برای جمع مبلغ تراکنش ها، = برای بررسی همسانی تراکنش ها)

مدیریت مشتریان (Customer)

- هر مشتری دارای شناسه، نام، مجموعه ای از حساب ها و دسترسی ها است
- ارتباط بین مشتری و حساب ها باید به صورت شیءگرا پیاده سازی شود

گزارش گیری (Report)

- نمایش وضعیت حساب ها و تراکنش ها به صورت مرتب و قابل فهم
- امکان جمع بندی تراکنش ها و موجودی ها

کلاس‌های الزامی

Account کلاس

- ویژگی‌ها:

- ;int id
- ;string ownerName
- ;double balance
- ;enum Type { SAVINGS, CHECKING, CREDIT } type

متدهای الزامی:

- deposit(double amount)
- withdraw(double amount)
- transfer(Account& to, double amount)
- printBalance()

Transaction کلاس

- ویژگی‌ها:

- int id
- enum Type { DEPOSIT, WITHDRAWAL, TRANSFER } type
- double amount
- string timestamp

مندهای الزامی:

- bool operator==(const Transaction&)
- Transaction operator+(const Transaction&)
- readable نمایش تراکنش‌ها به صورت

کلاس Customer

- ویژگی‌ها:

- ;int id
- ;string name
- ;vector<Account> accounts

متدهای الزامی:

- افزودن حساب جدید
- دسترسی به موجودی حساب‌ها
- چاپ گزارش مشتری

کلاس BankSystem

- ویژگی‌ها:

- ;vector<Customer> customers
- ;vector<Transaction> transactions
- ;static int totalAccounts

وظایف:

- مدیریت مشتریان و حساب‌ها
- ثبت و مدیریت تراکنش‌ها
- گزارش‌گیری کلی سیستم

نکات مهم طراحی

- استفاده هدفمند از **friend** برای دسترسی کنترل شده به داده های خصوصی
- **Operator Overloading** باید معنادار و منعکس کننده منطق سیستم باشد
- مدیریت **state** اشیاء و جلوگیری از وضعیت نامعتبر (مثلاً برداشت بیش از موجودی) الزامی است
- خوانایی، انسجام و طراحی قابل توسعه اهمیت دارد

Criterion	Description	Points
System & OOP Design	Clear architecture, separation of responsibilities, scalability	20
Correctness & Logic	Resource and interval management works, conflict detection correct	20
Operator Overloading	Operators reflect problem logic, not just syntax	10
Friend & Object State	Correct use of friend, safe state management	10
Function Overloading & Object Handling	Proper use of overloaded methods, pass by reference/const-ref	10
static & this	Meaningful use of static members, correct this usage / method chaining	10
Code Quality	Readability, formatting, naming, comments	10
Reporting & Output	Clear, structured schedule and resource reporting	10

تمرین سوم - بخش سوم | Turn-Based Strategy Game Engine Core

طراحی موتور بازی یکی از بهترین روش‌ها برای تمرین عمیق مفاهیم شیء‌گرایی است؛ زیرا نیازمند مدل‌سازی موجودیت‌ها، تعاملات پیچیده و قوانین پویا می‌باشد.

در این بخش، شما باید هسته یک بازی نوبتی (Turn-Based Strategy) طراحی کنید که شامل موارد زیر باشد:

- واحدهای بازی با قابلیت‌ها و رفتارهای متفاوت (حمله، دفاع، حرکت)
- نقشه یا محیط بازی برای استقرار واحدها و محدودیت‌های حرکتی
- سیستم نوبت‌دهی برای مدیریت ترتیب عمل واحدها
- سازوکار پایه مبارزه یا تعامل بین واحدها

سیستم طراحی شده باید:

- بر پایه تعامل اشیاء و چندریختی باشد
- از عملگرها برای بیان طبیعی رفتارهای بازی استفاده کند (مثلاً جمع آسیب‌ها، مقایسه قدرت واحدها)
- وضعیت بازی را به شکل شفاف و قابل گزارش مدیریت کند
- امکان گسترش و توسعه در آینده داشته باشد

اهداف اصلی سیستم

سیستم شما باید قابلیت‌های زیر را داشته باشد:

مدیریت واحدهای بازی (Unit)

- هر واحد دارای شناسه، نام، نوع و میزان سلامت (HP) است
- واحدها قابلیت‌های مختلفی دارند: حرکت، حمله، دفاع
- استفاده از چندریختی (polymorphism) برای رفتارهای متفاوت واحدها

مدیریت نقشه بازی (GameMap)

- نقشه به صورت grid یا collection از موقعیت‌ها مدل می‌شود
- موقعیت‌ها می‌توانند اشغال شده یا خالی باشند
- واحدها می‌توانند حرکت کنند و محدودیت‌های محیطی رعایت شود

سیستم نوبت‌دهی (TurnManager)

- تعیین ترتیب حرکت واحدها
- مدیریت گردش نوبت‌ها تا پایان بازی یا مبارزه
- استفاده از **static** برای وضعیت کلی بازی، مانند شماره نوبت جاری

تعامل و مبارزه (Combat)

- واحدها می‌توانند یکدیگر را هدف قرار دهند
- آسیب یا دفاع با استفاده از **operator overloading** بیان شود
- نتیجه مبارزه به‌روزرسانی HP و وضعیت واحدها است

گزارش‌گیری (GameState)

- نمایش وضعیت واحدها، موقعیت‌ها و نوبت‌ها
- خروجی خوانا و قابل پیگیری

کلاس‌های الزامی

کلاس Unit

- ویژگی‌ها:

- int id
- string name
- int hp
- int attackPower
- int defensePower

مندهای الزامی:

- `virtual void move(Position)`
- `virtual void attack(Unit&)`
- `bool isAlive() const`
- `printStatus()`

کلاس Position

- ویژگی‌ها:

int x ○

int y ○

متدهای الزامی:

- مقایسه موقعیت‌ها (>, ==)
- محاسبه فاصله بین دو موقعیت

کلاس GameMap

- ویژگی‌ها:

vector<vector<Unit*>> grid ○

وظایف:

- قرار دادن واحدها
- بررسی در دسترس بودن موقعیت‌ها
- چاپ وضعیت نقشه

کلاس TurnManager

- ویژگی‌ها:

;vector<Unit*> turnOrder ○

;static int currentTurn ○

وظایف:

- تعیین ترتیب نوبت‌ها
- حرکت به نوبت بعدی
- مدیریت وضعیت پایان بازی
-

کلاس CombatEngine

وظایف:

- مدیریت تعامل و مبارزه بین واحدها
- Overloading عملگرها برای بیان منطق حمله و دفاع
- بروزرسانی HP و وضعیت زنده بودن واحدها

نکات مهم طراحی

- مدیریت صحیح اشیاء و چندریختی الزامی است
- Overloading عملگرها باید منطق بازی را منعکس کند، نه صرفاً نمایش syntax
- استفاده static برای وضعیت کلی بازی، مانند شمارنده نوبت‌ها طراحی قابل توسعه و خوانا باشد
- گزارش‌گیری واضح از وضعیت بازی و واحدها الزامی است

Criterion	Description	Points
System & OOP Design	Clear architecture, separation of responsibilities, scalability	20
Correctness & Logic	Resource and interval management works, conflict detection correct	20
Operator Overloading	Operators reflect problem logic, not just syntax	10
Polymorphism & Inheritance	Proper virtual methods, overrides, unit specialization	15
Function Overloading & Object Handling	Proper use of overloaded methods, pass by reference/const-ref	10
static & this	Meaningful use of static members, correct this usage / method chaining	10
Code Quality	Readability, formatting, naming, comments	10
Reporting & Output	Clear, structured schedule and resource reporting	5

```
HW3-StudentNumber/
├── Section1_ResourceManagement/
│   ├── include/
│   └── src/
├── Section2_BankingEngine/
│   ├── include/
│   └── src/
└── Section3_TurnBasedGame/
    ├── include/
    └── src/
```

نحوه تحویل 📁

- تمامی فایل‌های مرتبط با تمرین، شامل سورس‌کد و هرگونه مستندات یا توضیحات تکمیلی، باید در یک پوشه با نام زیر قرار گیرند:

HW3-StudentNumber

- این پوشه را فشرده کرده و با فرمت **zip** ارسال نمایید.
- ارسال فایل تنها از طریق بخش "تمرین سوم" در کلاس درس، در سامانه کوئرا صورت می‌گیرد.

لطفاً پیش از اتمام مهلت مقرر، از آپلود صحیح فایل اطمینان حاصل فرمایید.